

pipeline-cli

Pipeline & Operator CLI Contract — Workshop Curriculum Platform

Feature: specs/001-workshop-curriculum-platform | **Phase:** 1 (Design & Contracts) | **Date:** 2026-09-01

Governs: the offline content pipeline and the operator control plane.

Derived from: [spec.md](#) FR-004/006/007/012/026–029/034/040, [plan.md](#) (corrected premises 1, 3, 4, 5), [research.md](#) D-TRANS-*, D-SEARCH-*, D-LLM-*, [data-model.md](#). **Companion contracts:** [passage-contract.md](#) (what ingest mints), [http-api.md](#) (what the server then serves).

Interfaces and behavioural guarantees only. No implementation, no migrations.

1. Exit codes — binding across this whole contract set

`.specify/memory/constitution.md`, **Honest Instruments**, verbatim:

A check that cannot run MUST report that it could not run. It MUST NOT report success, and it MUST NOT report failure. Three states are mandatory and distinct: 0 — the condition was checked and holds; 1 — the condition was checked and is violated; 2 — the condition could NOT be checked.

Collapsing state 2 into state 1 makes a broken tool accuse a healthy codebase. Collapsing it into state 0 makes a broken tool certify code nobody inspected. Both are release blockers. A missing credential, an unreachable service, a saturated backend, or a crashed helper are all state 2.

Every command in this document obeys it. No exceptions, no per-command variation.

1.1 The taxonomy

Exit	Name	Meaning
0	OK	The work was done, or the condition was checked and holds.
1	PROBLEM FOUND	A real defect in the content or the tree was detected. The instrument worked.
2	COULD NOT DETERMINE	The instrument itself could not run to a conclusion.

1.2 The pair that makes the distinction concrete

Situation	Exit	Why
The recording's SHA-256 mismatches the manifest	1	Computed successfully; the file is bad. A real problem, found.
The recording could not be read to compute a SHA-256	2	Nothing was determined about the file.

Same file, same command, opposite classes. Any command that returns 1 for the second case is accusing a healthy tree.

1.3 What is always 2

Non-exhaustive but binding by class — “*a missing credential, an unreachable service, a saturated backend, or a crashed helper are all state 2.*”

Condition	Note
A required binary is missing or unusable	Includes the <code>ffprobe</code> trap, §3.2
The Python venv, ASR model, or model file is absent	Premise 4: no ASR engine is installed on this host today
ollama / Lumen / the embedding backend is unreachable	
The embedding backend reports all embedding servers exhausted	Measured. Never 1 — the corpus is not at fault
An embed or search call exceeded its bounded timeout	
The distinct-vector probe fails (degenerate/repeated vectors)	The mode that put 758 duplicate vectors into this index on 2026-08-26
A lock is held by another run	
The evidence directory is unwritable	§2.5
Disk full, permission denied, OOM, RAM cap unsatisfiable	
A helper process crashed, was killed, or exited on a signal	
The command was interrupted (SIGINT/SIGTERM)	§1.5
Any unclassified non-zero exit inside the script	§1.4
A verification could not run (no human reference, no calibration set)	e.g. §4.2

1.4 Unclassified failures are 2, mechanically

`set -euo pipefail` makes an unexpected error exit with whatever the failing command returned — frequently 1 (a real problem, by our taxonomy) or 127 (command not found). Both would be lies.

Normative: every command installs an ERR trap and an EXIT trap that map any *unclassified* non-zero status to 2, and print the failing command and line. Only statuses set deliberately by the command's own classification logic may be 0 or 1.

Raw status	Mapped	Reason
126 (not executable), 127 (not found)	2	A missing tool determines nothing
128+N (killed by signal N)	2	A crashed helper is state 2 by name
Any other unclassified non-zero	2	If the command did not classify it, it did not determine anything
3–125	reserved, unused	Three states, not five

1.5 Interruption

SIGINT/SIGTERM $\Rightarrow$ flush the current checkpoint, write the run manifest, print INTERRUPTED with the resume command, **exit 2**. An interrupted run determined nothing about the whole job, even though completed chunks are preserved and will be skipped on resume (FR-029).

1.6 --json and the machine-readable result

Every command accepts `--json`. When given, **stdout carries exactly one JSON object and nothing else**; all human text goes to stderr.

```
{
  "command": "index",
  "run_id": "01JBX7QJ...",
  "state": "could_not_determine",      // "ok" | "problem_found" |
  "could_not_determine"
  "exit": 2,
  "reason": {
    "code": "embedding_backend_exhausted",
    "message": "all embedding servers exhausted after failover:
context deadline exceeded",
```

```

    "remedy": "wait for the running index rebuild to finish, or
run with --lexical-only"
  },
  "findings": [],                                // populated when exit==1:
one entry per real problem found
  "partial": { "lexical": "ok", "semantic": "failed" },
  "evidence_dir": "workshop/_evidence/index/
2026-09-01T10-14-02Z-01JBX7QJ...",
  "started_at": "...", "ended_at": "...", "duration_s": 41.2
}

```

state and exit **MUST** agree. A gate asserts the mapping over every command (§5, G-CLI-1).

2. Conventions common to every command

2.1 Location and invocation

Layer	Location	Shape
Control plane	workshop/scripts/ {build,start,stop,restart,status,verify}.sh	Script names mirror platform/scripts/{build,ingest,resta (FR-013). The direct — see the path decis
Shared wrapper library	workshop/scripts/_common.sh	Configuration resolu (\$1.4), evidence path reference's own _com executed.
Pipeline wrappers	workshop/scripts/{transcribe,verify-accuracy,ingest,index,crossref,redact,add-chapter}.sh	Bash entry points
Pipeline implementation	workshop/pipeline/ {transcribe,ingest,crossref}/	Python/Go called by
Orchestration consumer	workshop/platform/orchestration/	The Go module that digital.vasic.conta (§11.4.76(2)) and bui adapter into worksho script layer — §4.8
Pre-existing, reused	workshop/scripts/{archive-videos,extract-videos,install-hooks,self-test}.sh	Invoked, never rein

Path decision — `workshop/scripts/`, **not** `workshop/platform/scripts/` (settled 2026-09-01; recorded because two documents in this set had drifted apart on it, and because the justification this table previously gave cited the wrong reference directory).

The correction first. This table used to justify the control plane as mirroring `ai_interviewing's scripts/`. That citation was wrong, and is withdrawn. Measured:

```
ai_interviewing/scripts/          build-all.sh  export-doc.sh
render-diagrams.sh  validate-exports.sh
ai_interviewing/platform/scripts/ _common.sh  build.sh
ingest.sh  restart.sh  start.sh  status.sh  stop.sh
```

`ai_interviewing/scripts/` is **documentation tooling**. The real reference control plane is `ai_interviewing/platform/scripts/`. Note what survives the correction: the six script *names* this table already listed match `ai_interviewing/platform/scripts/` exactly. Only the directory was mis-cited — the mirror claim was right about the thing that matters and wrong about where to look.

The decision. The control plane lives at `workshop/scripts/`, one script directory for the whole module. Three reasons, in decreasing weight:

1. **workshop/scripts/ already exists and is load-bearing.** It holds `{archive-videos,extract-videos,install-hooks,self-test}.sh` plus `git-hooks/` — the reassembly and integrity machinery FR-007 is satisfied by **invoking**, and which this contract marks *invoked*, *never reimplemented*. Those cannot move without breaking `install-hooks.sh` and the tooling that already calls them.
2. **A split would make FR-013 worse, not better.** Putting the control plane under `workshop/platform/scripts/` while the pipeline wrappers and the pre-existing scripts stay at `workshop/scripts/` yields two script directories in one module and no rule for which holds what. FR-013 asks that someone familiar with one module can navigate the other; one obvious directory serves that better than a structural echo that fragments the module.
3. **The reference's nesting is not transferable anyway.** `ai_interviewing` is a platform-and-nothing-else module, so its control plane naturally sits under `platform/`. The workshop additionally owns a *pipeline* layer and *source material*, and its wrappers span all three — they are module-scoped, not platform-scoped. `workshop/platform/` is the built product (`backend/`, `frontend/`, `bin/`, `orchestration/`), which is exactly what a script that drives the pipeline is not part of.

FR-013 is therefore satisfied by **name and exit-semantic parity**, which is what a person actually carries between modules, rather than by directory nesting. Any document in this set invoking `workshop/platform/scripts/...` is stale against this decision and is corrected to `workshop/scripts/`.

`extract-videos.sh` already verifies per-part, archive and extracted-video hashes against the `video-archive-manifest v1`. FR-007 is therefore satisfied by **invoking** it. Note its current contract is two-valued (`die() → exit 1`); §4.1 specifies the wrapper that classifies its failure into 1 vs 2 rather than editing a script that other tooling depends on.

Every command supports `--help` (exit 0), `--version` (exit 0), `--dry-run`, `--json`, `-v`.

2.2 Environment overrides (Environment Adaptability)

The constitution requires that no file freeze an assumption about the host, and that every derived value be overridable. Every variable below is **derived** when unset; setting it overrides detection.

Variable	Derived from	Governs
WORKSHOP_ROOT	<code>git rev-parse --show-toplevel</code> then <code>/workshop</code>	Module root
WORKSHOP_CHAPTERS_DIR	<code>\$WORKSHOP_ROOT/chapters</code>	Source material
WORKSHOP_CURRICULUM_DIR	<code>\$WORKSHOP_ROOT/curriculum</code>	Published output, <code>passages.jsonl</code>
WORKSHOP_EVIDENCE_DIR	<code>\$WORKSHOP_ROOT/_evidence</code>	FR-040
WORKSHOP_LOCK_DIR	<code>\$WORKSHOP_ROOT/.locks</code>	§2.6
WORKSHOP_FFMPEG / WORKSHOP_FFPROBE	capability probe, §3.2	Media tooling
WORKSHOP_PYTHON / WORKSHOP_VENV	<code>\$WORKSHOP_ROOT/.venv</code>	ASR runtime
WORKSHOP_ASR_MODEL	<code>large-v3-turbo</code>	D-TRANS-1
WORKSHOP_ASR_COMPUTE_TYPE	<code>int8</code>	CPU-only
WORKSHOP_ASR_THREADS	<code>nproc</code> minus a reserved core	
WORKSHOP_CHUNK_SECONDS	<code>300</code>	D-TRANS-4

Variable	Derived from	Governs
WORKSHOP_DB	\$WORKSHOP_CURRICULUM_DIR/ passages.db	
WORKSHOP_GENERATIONS_DIR	\$WORKSHOP_CURRICULUM_DIR/ generations	D-SEARCH-5
WORKSHOP_PORT / WORKSHOP_BIND	8080 / 127.0.0.1	\$4.8; loopback by default (D1)
WORKSHOP_MAX_RAM_PCT	60	Constitution, Quality Over Speed
WORKSHOP_ANSWERING_PROVIDER / _ENDPOINT / _MODEL / _LOCALITY	answering.yaml	D-LLM-1
OLLAMA_HOST	http://127.0.0.1:11434	Honoured as a fallback for endpoint when provider: ollama, mirroring scripts/lumen- reindex.sh so an operator learns one pattern
LUMEN_*	existing repo convention	Semantic backend

Resolution order is fixed and identical everywhere: environment → config file → live probe → documented fallback. This is the order scripts/lumen-reindex.sh already establishes in this repository; copying it means one pattern, not two.

Derived values must print their inputs. The constitution requires that “*a tuned value MUST be computed from measured host facts and MUST print its inputs so the arithmetic can be audited.*” Each command prints, at -v, the host facts it measured and the arithmetic it performed — e.g. MemTotal=32,614 MiB × 60% = 19,568 MiB cap; model needs ~2,100 MiB; threads=7 (nproc 8 – 1).

2.3 Output discipline

Human progress and diagnostics → **stderr**. Machine results → **stdout** (only under `--json`). This keeps `cmd --json | jq` correct while a human still sees progress.

2.4 Read-only guarantee on source material (FR-006)

#	Rule
S1	Source material — recording, archive parts, notes PDF, supplied diagrams and code — is opened <code>O_RDONLY</code> .
S2	Before starting, each command records (<code>size</code> , <code>mtime</code> , <code>inode</code>) of every source it will read. On exit it re-checks them. A change ⇒ exit 1 (a real problem: something modified the sources).
S3	Derived artifacts are written only under <code>chapters/NN/transcript/</code> , <code>curriculum/</code> , <code>_evidence/</code> and <code>.locks/</code> .
S4	No command writes to <code>chapters/NN/*.mp4</code> , <code>*.tar.gz.*</code> , <code>*.sha256</code> , or <code>*.pdf</code> .

Gate G-CLI-2: run the full pipeline; assert every source file's (`size`, `mtime`, `inode`) is unchanged. **Paired mutation:** have transcribe rewrite the notes PDF in place. Gate must FAIL.

2.5 Evidence (FR-040, SC-018)

Every run writes to `$WORKSHOP_EVIDENCE_DIR/<command>/<UTC-timestamp>-<run_id>/:`

File	Contents
<code>manifest.json</code>	Command, argv, resolved environment (secret names only, never values), host facts, tool versions and their capability-probe results, start time

File	Contents
result.json	The --json object of §1.6
stdout.log, stderr.log	Full streams
findings.jsonl	One line per problem found (exit 1)
Command-specific	e.g. coverage.json, accuracy.json, generation.json

#	Rule
E1	Evidence is written on every outcome — 0, 1 and especially 2. A run that could not determine anything is the run whose evidence matters most.
E2	If the evidence directory cannot be created or written, the command exits 2 immediately, before doing any work , printing the reason to stderr. It does not attempt to write evidence about failing to write evidence, and it does not proceed unrecorded.
E3	Evidence is committed alongside the change it describes, so it is reviewable at the commit that produced it — mirroring the repository’s existing _tests/ evidence/ practice.
E4	Secrets are never written. api_key_env holds the name of an environment variable; the manifest records the name, never the value, never a file path.

2.6 Locking

One exclusive lock per chapter: flock on \$WORKSHOP_LOCK_DIR/<slug>.lock, plus a global lock for index.sh --swap.

Condition	Exit
Lock acquired	proceed

Condition	Exit
Lock held by a live process	2, reason.code: "lock_held", with the holder's pid and command
Lock file present, holder not alive	Stale lock is reclaimed, and the reclamation is recorded in the manifest
Lock directory unwritable	2

A held lock is never 1. Another run being in progress is not a defect in the content.

2.7 No CI (FR-034, SC-014, constitution §11.4.156)

#	Rule
N1	These commands are local only . No command in this contract creates, references or requires a <code>.github/workflows/*.yaml</code> file anywhere, including inside <code>workshop/</code> .
N2	<code>verify.sh</code> (§4.9) is the aggregation point the umbrella's local pre-push hook calls. It is a script, not a workflow.
N3	Introducing an active workflow is a release blocker. <code>scripts/pre-push-gates.sh</code> gate E already enforces this at the umbrella root; the workshop's own tree is covered by the same fleet check, which derives the owned set from <code>helix-deps.yaml</code> .

2.8 Memory (constitution, Quality Over Speed)

Each long-running command computes a cap from `MemTotal` × `WORKSHOP_MAX_RAM_PCT` and prints the arithmetic. It reduces concurrency to fit. If the job cannot fit even at minimum concurrency ⇒ exit 2, reason.code: "insufficient_memory" — not a crash, and not a silent overcommit.

2.9 Never edit a running script

The constitution states it plainly: “*Never edit a shell script while it is executing — bash reads scripts lazily and a live edit makes the running shell execute garbage.*” Long-running commands (`transcribe.sh`, `index.sh`, `add-chapter.sh`) record their own file hash in the manifest at start and re-check it at each checkpoint. A change $\Rightarrow$ abort with exit 2, `reason.code: "script_modified_while_running"`.

3. Preflight — the checks that run before any command does work

3.1 Structure

Preflight is a shared routine. It classifies every finding as **2 (tooling)** or **1 (content)** and reports **all** findings, not the first (FR-028’s “report precisely what is missing” applies to tooling too).

3.2 The `ffprobe` capability probe — a named check, because of a measured trap

Corrected premise 3 (`plan.md`): `<home>/bin/ffprobe` is a **symlink to Playwright’s `ffmpeg` binary**. It accepts `--version` — so a naive probe reports it present — and rejects `-show_format` with “*Unrecognized option*”, which reads like a bad-argument error rather than a missing tool. Both `ffmpeg` and `ffprobe` resolve into `~/.cache/ms-playwright/ffmpeg-1011/`, an `npx`-managed cache that `npx playwright uninstall` would remove.

Normative:

#	Rule
P1	<code>ffprobe --version</code> succeeding is NOT acceptance and MUST NOT be used as the probe. The probe runs
P2	<code>ffprobe -v quiet -print_format json -show_format -show_streams <fixture></code> against a tiny committed media fixture and

#	Rule
	requires exit 0 and parseable JSON containing a format object.
P3	The probe likewise exercises <code>ffmpeg</code> by decoding the fixture to a null sink.
P4	Failure $\Rightarrow$ exit 2, <code>reason.code</code> : " <code>ffprobe_unusable</code> ", with the resolved path, the symlink target, and the remedy.
P5	The resolved path is recorded in the manifest. If it resolves inside a browser-automation cache, the command prints a warning: a production content pipeline must not depend on an npx-managed cache.

Gate G-CLI-3: point `WORKSHOP_FFPROBE` at Playwright's `ffmpeg` binary; assert exit 2 with `ffprobe_unusable`. **Paired mutation:** change the probe to `ffprobe --version`. Gate must FAIL — it will report the tool as present.

3.3 Other preflight checks

Check	Failing exit	Note
Python venv exists and imports <code>faster_whisper</code>	2	Premise 4: no ASR engine is installed on this host today; <code>/usr/bin/whisper</code> is <code>whisper-1.3.1-alt1</code> , an unrelated microphone-loopback GUI. <code>import whisper</code> fails.
ASR model file present and hash matches	2	
SQLite build has FTS5	2	Verified available in modernnc.org/sqlite , already in the

Check	Failing exit	Note
Embedding backend reachable and answering	2	reference module's <code>go.mod</code> A liveness probe alone is insufficient — <code>health_check</code> returned OK 2 ms before <code>semantic_search</code> failed with all embedding servers exhausted. The probe must issue a real bounded embed.
Chapter directory exists	1	Content
Chapter material set complete	1	FR-028 — §4.7
Evidence dir writable	2	E2
Free disk $\geq$ required	2	Decode alone produces a 211 MiB WAV; reassembly needs 1.8 GB

4. Command contracts

4.1 `transcribe.sh` <chapter-slug>

Traces: FR-001, FR-002, FR-003, FR-005, FR-006, FR-007, FR-029, SC-001, SC-003.

Arguments

Argument	Default	Meaning
<chapter-slug>	required	
<code>--resume</code>	off	Skip chunks whose output exists and whose embedded

Argument	Default	Meaning
		input hash matches (FR-029)
--from-parts	auto	Invoke extract-videos.sh first if the reassembled file is absent
--chunk-seconds N	300	D-TRANS-4
--model M	large-v3-turbo	
--threads N	derived	
--sample-seconds N	—	Calibration mode: transcribe only the first N seconds (this is the run that closes U1/U2/U3)
--dry-run	off	Preflight + partition only; write chunks.json, transcribe nothing

Side effects — all under chapters/NN/transcript/:

```

transcript/work/audio.wav      16 kHz mono, ~211 MiB, decoded
ONCE and hashed
transcript/work/vad.json      speech/non-speech regions,
stamped with the WAV hash
transcript/work/chunks.json    deterministic partition; each
entry has index, abs start/end, hash
transcript/work/chunks/NNN.json per-chunk output – the checkpoint
transcript/machine.jsonl      assembled machine passages
(immutable once written)
transcript/transcript.machine.md renderable machine transcript
transcript/coverage.json      the SC-001 arithmetic identity
$WORKSHOP_EVIDENCE_DIR/transcribe/<ts>-<run_id>/

```

Idempotency & resume: the partition is deterministic — identical WAV hash $\Rightarrow$ identical chunks.json. Resume recomputes VAD and partition, then skips any chunk whose output exists **and** whose embedded input hash matches. Everything else is redone. There is no state beyond the files themselves. Each chunk is written .tmp $\rightarrow$ fsync $\rightarrow$ rename(); **the atomic rename is the checkpoint**, so a crash leaves each chunk file either absent or complete, never half-written. A half-

written chunk that parsed as valid JSON would be silent, plausible data loss — which is why the rename, not a lock or a database, is the mechanism.

Progress (FR-029): chunks completed / 24, audio-seconds completed / 6928.75, elapsed, and a projected finish **derived from the measured rate so far — never from the D-TRANS-1 estimate** (which is itself UNVERIFIED, §6 U2). Written to the run manifest as it goes, so an interrupted run leaves a readable record of how far it got.

Coverage proof (SC-001): $\bigcup$ passage spans $\cup$ $\bigcup$ VAD non-speech spans == $[0, \text{duration}_s)$ exactly. Any second belonging to neither is an **unexplained gap**. The eight measured long silences (4552.33, 4626.42, 4872.28, 5009.90, 5025.19, 5998.99, 6063.45, 6925.35 s) are a ready-made VAD fixture: the detector must independently classify all eight as non-speech, and gap 8 is the recording's trailing tail — the assembled transcript must terminate with an accounted-for non-speech region, which is the most likely place for the coverage assertion itself to be wrong.

Uncertainty (FR-003): uncertain is set from the engine's avg_logprob, no_speech_prob and compression_ratio. faster-whisper is chosen precisely because it returns them; without a confidence signal FR-003 has nothing to key on and becomes editorial opinion. Text is **never** invented for an unclear segment.

Speakers (FR-005): speaker is left null with speaker_source: "unattributed". No diarization is run: the audio is dual-mono (L-R = -90.3 dB) and uniformly AGC-compressed, so both cues that make diarization tractable are measurably absent.

Exit codes

Exit	Conditions
0	All chunks transcribed, coverage identity holds, sources unchanged
1	Unexplained coverage gap; a VAD-declared silence contradicting a measured long silence; source (size, mtime, inode) changed during the run; extract-videos.sh reported a checksum mismatch

Exit	Conditions
2	ffmpeg/ffprobe unusable (§3.2); venv or model missing; embedding/ASR helper crashed; disk full; RAM cap unsatisfiable; lock held; interrupted; extract- videos.sh could not read a part; script modified while running

Gate G-CLI-4: delete one chunk’s output and assert the coverage check reports exit 1. **Paired mutation:** make coverage tolerate gaps under a threshold. Gate must FAIL. (*This is the mutation proof D-TRANS-4 names.*)

4.2 verify-accuracy.sh <chapter-slug>

Traces: FR-004, FR-033, SC-002, SC-013.

Arguments

Argument	Default	Meaning
--reference <path>	required	Blind human transcription of the sampled windows
--windows N	30	≥30 stratified 30-second audio-timeline windows
--seed N	recorded	Seeded so the sample is reproducible
--normaliser <path>	frozen	Its SHA-256 is recorded and must match the frozen hash
--min-accuracy X	unset	When set, a WER above 1 – X is a finding

Method (D-TRANS-3): word error rate against a blind human reference over seeded, stratified **audio-timeline** windows — not sampled machine passages. Sampling passages makes whole-region deletions structurally invisible: a dropped span produces no passage to

sample, so accuracy is biased upward exactly where the transcript is worst. Thirty 30 s windows span well over 30 passages, so SC-002's wording is satisfied on its own terms while the bias is removed. Companion metrics: coverage-gap rate (SC-001), timestamp error median/p95 against the 5 s bound (SC-003), speaker accuracy (FR-005).

Side effects: chapters/NN/transcript/accuracy.json, plus evidence.

Exit codes

Exit	Conditions
0	WER measured and written; if --min-accuracy was given, it is met
1	WER measured and below --min-accuracy; or a companion metric breached (e.g. timestamp p95 > 5 s)
2	--reference missing or unreadable ; normaliser hash mismatch; fewer windows available than requested; audio unreadable

The first 2 case is the most important line in this section: **the absence of a human reference is 2, never 0**. A command that cannot measure accuracy must not report that accuracy is fine. Until this command has run, GET /api/chapters/{c}/accuracy reports measured: false, wer: null (<http://api.md> §3.12).

Gate G-CLI-5: run with --reference pointing at a nonexistent file; assert exit 2. **Paired mutation:** return 0 with wer: 0.0 when the reference is absent. Gate must FAIL.

4.3 ingest.sh [<chapter-slug>]

Traces: FR-016, FR-027, FR-037, FR-038, SC-016.

Arguments: --write-anchors (default on for workshop/-owned text), --no-write-anchors, --kinds transcript,doc_section,code,diagram, --corpus <path> (D2: workshop/ plus the vasic monorepo), --dry-run, --check-idempotent.

Behaviour: implements the matching algorithm and idempotency contract of passage-contract.md §8 exactly. Nothing here restates it; the two must not drift.

Side effects: curriculum/passages.jsonl (sorted, byte-stable), curriculum/passages.db, anchors written back into workshop/-owned text sources, sidecars for anchorless formats, evidence.

Idempotency (FR-027): a second run over unchanged input mints zero pids, writes zero anchors, produces a byte-identical passages.jsonl, leaves every source byte-identical, creates no new generation, and exits 0. --check-idempotent performs the second run into a scratch tree and diffs, reporting exit 1 on any difference.

Exit codes

Exit	Conditions
0	Ingest complete; registry consistent
1	Duplicate anchor (R1b); foreign anchor (R1c); unaliased rename of workshop/-owned code (R4); non-idempotent second run; a pid collision; a passage violating a field invariant (F1–F7)
2	Registry unreadable or locked; symbol source unavailable (§6 P-U1); disk full; interrupted

4.4 index.sh

Traces: FR-014, FR-015, FR-016, FR-020, FR-027, SC-005, SC-006.

Arguments

Argument	Default	Meaning
--chapter S	all	Scope Build only the FTS5 leg. The honest escape hatch when the embedding
--lexical-only	off	

Argument	Default	Meaning
		backend is saturated.
--semantic-only	off	
--verify-only	off	Run the verification gate against an existing building generation; swap nothing
--swap	on	Promote verified → live by atomic swap
--no-swap	off	Build and verify; leave promotion to a later run
--timeout-ms N	5000 per call	Bounded — an unbounded query is how a 10-minute stall happens

Behaviour (D-SEARCH-5): builds a **new** generation; never mutates the live one. Only a verified generation may become live. Readers never see building. A generation failing its gate is discarded and never served — which is what keeps FR-020 honest. Verification inputs include `pid_count` and the `root_hash` over member `content_hashes`.

This discipline exists because a half-written readable index was **measured**: during the live rebuild, chunks moved 58,734 → 58,744 while `last_indexed_at/root_hash` still advertised the previous generation.

Interactive capacity reservation: index runs must leave embedding capacity for interactive queries. This is a hard requirement, not a nicety — `scripts/ollama-tune.sh` records that with `OLLAMA_NUM_PARALLEL` resolving to 1, “*a single embed went from 0.74 s to a >90 s client timeout, which stalled indexing entirely*”, with the note “*queue depth is not the defect — serialisation is.*” Without reservation, SC-006 fails on every chapter ingest — precisely when people are using the system.

Exit codes

Exit	Conditions
0	

Exit	Conditions
	New generation verified and live, or already current with nothing to do
1	Verification gate failed — the generation is discarded. A real problem, found: e.g. pid_count mismatch, a member pid absent from the registry, a redacted passage present in the index
2	Embedding backend unreachable, exhausted , timed out, or returning degenerate vectors; Lumen unreachable; no verified generation exists to serve; global swap lock held; disk full; interrupted

The line that must not be blurred: all embedding servers exhausted is **exit 2**, never 1. The corpus is not at fault when the backend is saturated. Reporting 1 there makes a broken backend accuse a healthy curriculum — the exact conflation the Honest Instruments principle names.

Partial outcomes: if `--semantic-only` fails but a lexical generation is usable, the exit is still 2 (the requested work did not complete), and `result.json` carries `"partial": {"lexical": "ok", "semantic": "failed"}` so the operator learns that search will degrade rather than die.

Gate G-CLI-6: point the embedding endpoint at a closed port; assert exit 2 with `embedding_backend_exhausted` **and** that the previously live generation is still live and serving. **Paired mutation:** map backend failure to exit 1. Gate must FAIL.

4.5 crossref.sh

Traces: FR-018, SC-016.

Arguments: `--chapter S`, `--min-score X`, `--max-per-passage N` (default 20), `--rebuild-derived`, `--check-cycles`.

Behaviour: rebuilds origin: "derived" edges for the target generation. **authored edges are never touched** — they are content, not derivation ([passage-contract.md §7.4 X4](#)). Self-references are rejected. Traversal is cycle-checked.

Exit codes

Exit	Conditions
0	Edges built; no cycle among authored edges
1	A cycle among authored edges (a content defect a human introduced); an edge whose endpoint is not in the registry; a self-reference in authored content
2	Embedding backend unavailable for similarity scoring; registry unreadable; interrupted

4.6 `redact.sh`

Traces: FR-039, SC-016.

Arguments: `--pid P` (repeatable), `--pids-file F`, `--reason CODE` (required), `--by NAME` (required), `--unredact`, `--review-only`.

Behaviour: appends to the redactions log, materialises `passages.redacted`, marks affected stored answers withdrawn, and marks the live generation as requiring a rebuild. Propagation rules R1–R7 of [passage-contract.md §7.3](#) apply in full — index, cross-references and stored answers, not merely the rendered transcript.

`--review-only` records the FR-039 redaction review for a chapter without redacting anything. Recording “*none required*” is an explicit, valid decision; **skipping the review is not**. The review artifact is `chapters/NN/redaction-review.json` and must be newer than the transcript it reviews.

Exit codes

Exit	Conditions
0	Redaction (or review) recorded

Exit	Conditions
1	A supplied pid is not in the registry; a redaction is requested for a pid already redacted with a different reason; the review is stale relative to the transcript
2	Registry unwritable; the index rebuild required to make the redaction effective could not be started; interrupted

4.7 add-chapter.sh <chapter-slug> — the FR-026 procedure

Traces: FR-026, FR-027, FR-028, FR-029, FR-039, SC-011, SC-016.

This is the single documented, repeatable procedure. It requires **no code changes** to add a chapter — a new chapter is a new directory plus a manifest.

Required inputs (this list is the contract; anything else is optional):

chapters/<NN>/chapter.yaml	ordinal, slug, title, summary
chapters/<NN>/<recording>.tar.gz.NNN	size-bounded parts
chapters/<NN>/<recording>.sha256	video-archive-manifest v1
chapters/<NN>/materials/	optional supporting material

Stages, in order: preflight → extract → transcribe → verify-accuracy → ingest → index → crossref → redaction-review → publish.

Arguments: --resume, --only <stage>, --skip <stage>, --from <stage>, --dry-run, --json, --check-idempotent.

FR-028 — incomplete material sets

Preflight enumerates **every** missing or malformed item, not the first, and separates the two classes:

```
{ "state": "problem_found", "exit": 1,
  "findings": [
    { "code": "missing_file",    "path": "chapters/02/
chapter.yaml" },
    { "code": "missing_field",  "path": "chapters/02/
chapter.yaml", "field": "summary" },
    { "code": "parts_incomplete", "expected": 36, "present": 34,
```

```
"missing": ["...tar.gz.021", "...tar.gz.030"] }
] }
```

- Missing **materials** $\Rightarrow$ exit 1. Determined: the chapter is incomplete.
- Missing **tooling** $\Rightarrow$ exit 2. Determined nothing about the chapter.

A partial chapter is never published as complete. Publication requires, all of them:

#	Publish precondition
B1	<code>coverage.unexplained_gap_s == 0</code>
B2	<code>accuracy.json</code> exists — i.e. <code>verify-accuracy.sh</code> has actually run (FR-004). Its <i>value</i> may be below target; its <i>absence</i> blocks.
B3	Every passage has a pid in <code>passages.jsonl</code>
B4	The generation containing the chapter is live
B5	<code>redaction-review.json</code> exists and is newer than the transcript (FR-039)
B6	No source file changed during the run (S2)

Failing any of B1–B6 $\Rightarrow$ exit 1, chapter remains transcribed, not published.

FR-027 — idempotency. A second run on the same chapter MUST:

#	Assertion
J1	Exit 0
J2	Mint zero pids, write zero anchors
J3	Leave <code>passages.jsonl</code> byte- identical
J4	Leave the published curriculum tree byte-identical, except for new evidence directories, whose timestamps and run ids necessarily differ

#	Assertion
J5	Create no new index generation
J6	Produce no duplicate chapter, section, passage or cross-reference

--check-idempotent performs the second run and asserts J1–J6, reporting exit 1 on any breach.

Gate G-CLI-7: run `add-chapter.sh` twice on a small synthetic chapter; assert J1–J6. **Paired mutation:** make ingest mint whenever no anchor was read this run. Gate must FAIL on J2, J3 and J5 simultaneously.

Gate G-CLI-8: remove `chapter.yaml` and two archive parts; assert exit 1 with **three** findings enumerated and nothing published. **Paired mutation:** report only the first finding and publish anyway. Gate must FAIL.

SC-011 (a maintainer adds a chapter in under 30 minutes of hands-on time) is about *hands-on* time. Transcription wall-clock is 40–80 minutes idle by estimate — **UNVERIFIED** (§6 U2) and roughly doubled under current load. The command is therefore resumable and backgroundable, and --json progress is machine-readable so the maintainer is not sitting at a terminal.

4.8 Control plane — build / start / stop / restart / status

Traces: FR-012, FR-013, FR-025, SC-004, D1, §11.4.76, §11.4.161.

Rewritten 2026-09-01. An earlier revision of this section specified **native** launch — “*start.sh/stop.sh ... run the built binary natively, as ai_interviewing does*” — as the forced consequence of C-U6’s since-withdrawn “not a submodule of this tree”. That premise is withdrawn (§7.1) and so is the native specification that rested on it. **The control plane is containerised.** This is not a preference this document is free to re-litigate: the operator requires container operation, and §11.4.76 makes `vasic-digital/containers` the only legal way to provide it. The two point the same way.

4.8.0 The orchestration seam — what is consumed and what is written here

submodules/containers (digital.vasic.containers, gitlink 4dab992) supplies, as a **Go API**: runtime auto-detection with **podman first and rootless** (§11.4.161 — the host has podman 5.7.1 rootless and no docker) in pkg/runtime; compose orchestration that resolves the compose CLI to podman-compose and suppresses the docker-only --wait flag in pkg/compose; and TCP/HTTP/gRPC health checkers in pkg/health. §11.4.76(3) names exactly these packages as the things a consuming project invokes.

cmd/boot is NOT the seam, and specifying it would have specified a silent false pass. An earlier revision of this section said the wrappers “invoke cmd/boot”. Measured at gitlink 4dab992 in submodules/containers/cmd/boot/main.go:

Measured	Consequence for this contract
Its whole flag set is --env, --project, --timeout, --help. There are no up / down / status subcommands and no compose-file flag at all .	The lifecycle verbs and the compose file this control plane needs cannot be expressed through it.
It hardcodes a single endpoint map, {"helixagent": localhost:7061} (main.go:148–153), with no flag to change it.	It boots someone else's service, not this stack.
Its boot.NewBootManager is constructed with runtime, logger, projectDir, distributor, hostManager and scheduler — no orchestrator and no health checker .	Nothing brings a compose project up and nothing verifies health.
runBoot returns nil after BootAll, so the process exits 0 having started nothing .	A wrapper trusting that exit code reports a healthy stack that does not exist.

That last row is decisive: cmd/boot is a **green instrument over an absent system**, the precise bluff shape §11.4.76(5) and the constitution's anti-bluff rules exist to catch. A contract that named it would have mandated the failure its own G-CLI-15 forbids. This is

recorded rather than quietly routed around, because the next reader will otherwise reach for `cmd/boot` for the same reason the earlier revision did — it is the obviously-named entry point.

The seam is therefore the Go API, reached through a consuming adapter. `workshop-boot` (built from `workshop/platform/orchestration/cmd/workshop-boot`, module `.../platform/orchestration`, which consumes `digital.vasic.containers` via the §11.4.76(2) `replace`) imports `pkg/compose`, `pkg/health`, `pkg/runtime` and `pkg/logging` and exposes the verbs this control plane actually needs: `up`, `down`, `status`, `probe`. It **does not** shell out to `podman`.

This is consumption, not reimplementation, and the distinction is not a matter of framing: the adapter contains no runtime detection, no `compose` driving, and no health-check logic of its own — all three are called in the submodule. What it adds is argument surface and the §1 exit mapping.

What this feature writes, and **all** it may write:

Layer	Written here?	Why that is §11.4.76-legal
Runtime detection, <code>compose</code> orchestration, health checking	No — consumed from <code>pkg/runtime</code> , <code>pkg/compose</code> , <code>pkg/health</code>	§11.4.76(1),(4): reimplementation is the violation
Go module consuming <code>digital.vasic.containers</code> via <code>replace</code> (§11.4.76(2))	Yes	The prescribed development-time consumption form §11.4.76(3)
The <code>workshop-boot</code> adapter exposing <code>up/down/status/probe</code> over that API	Yes	prescribes invoking the <code>boot/compose/health</code> APIs; a CLI face over them adds no orchestration
A <code>compose</code> file describing <i>this</i> stack's services	Yes	<code>ComposeProject.File</code> accepts an arbitrary <code>compose</code> file; supplying data to a consumed API is consumption
Thin <code>bash</code> wrappers that invoke <code>workshop-boot</code>	Yes	No usable <code>bash</code> or CLI boot entry point exists upstream.

Layer	Written here?	Why that is §11.4.76-legal
		Anything beyond thin wrapping must be contributed upstream, not grown locally

Upstream debt this creates, stated rather than left implicit. `cmd/` boot’s four defects above are upstream defects. §11.4.76(4) makes fixing them there — a `compose-file` flag, lifecycle verbs, an orchestrator and health checker on the `BootManager`, and a non-zero exit when nothing booted — the correct long-run remedy, with the adapter narrowing as they land. `workshop/platform/upstream-contributions/` is where that work is staged. The adapter is legal today because it consumes rather than reimplements; it is not a licence to keep growing locally what belongs upstream.

Explicitly forbidden, and named so it cannot be reintroduced by drift: a hand-rolled `Containerfile-plus-podman` run stack, a bash *or Go* reimplementation of health polling, backoff or compose driving inside `workshop/`, and any direct `podman/podman-compose` invocation that bypasses `workshop-boot` to bring the stack up or down. Direct `podman` calls are permitted **only** for read-only post-mortem diagnostics on a failure path (for example tailing a dead container’s logs), never for lifecycle.

Each bash wrapper is a *translator*, not a *manager*: it resolves configuration per §2.2, invokes `workshop-boot`, and maps the result onto the §1 three-valued exit taxonomy. That mapping is the only orchestration-adjacent logic the bash layer owns.

Additional §2.2 variables this seam requires — derived when unset, overridable, same resolution order as everywhere else:

Variable	Derived from	Governs
<code>WORKSHOP_COMPOSE_FILE</code>	<code>\$WORKSHOP_ROOT/platform/compose.yml</code>	The stack definition handed to <code>workshop-boot</code>
<code>WORKSHOP_PROJECT_NAME</code>	<code>workshop_curriculum</code>	Compose project label; scopes every lifecycle call
<code>WORKSHOP_BOOT_BIN</code>		The built adapter over <code>pkg/compose</code> +

Variable	Derived from	Governs
	<code>\$WORKSHOP_ROOT/ platform/bin/ workshop-boot</code>	<code>pkg/health + pkg/ runtime</code>
<code>WORKSHOP_STATE_FILE</code>	<code>\$WORKSHOP_ROOT/ platform/run/ server.json</code>	Resolved endpoint of the running stack — replaces the pidfile a native design would have used
<code>WORKSHOP_STACK_WAIT</code>	30	Seconds workshop- boot waits for health, via <code>pkg/ health</code>

The pidfile is gone. A container’s liveness is not a PID on this host, so every exit-code condition below that previously read “pidfile” now reads against the container state and the resolved endpoint. Where the old text said `/proc` unreadable, the new condition is that the container runtime could not be reached or its report could not be parsed. The **taxonomy is unchanged**; only the evidence it reads has changed.

build.sh

Builds, in this order: the `workshop-boot` adapter into `$WORKSHOP_BOOT_BIN`; the Go backend and the Angular frontend into `workshop/platform/bin/`; then the container image(s) the compose file names, through the containers submodule rather than a direct podman build.

Exit	Conditions
0	Everything built
1	Compile, test or image-build failure — a real problem in the code or the compose definition
2	Toolchain missing (Go, Node, no container runtime detected); network needed and unreachable; disk full

“No container runtime detected” is emphatically a 2, not a 1: the host could not be assessed, which is not a finding about the code.

`start.sh`

Argument	Default	Notes
<code>--port N</code>	8080	Published port on the host Loopback by default (D1). A non-loopback bind requires an explicit <code>--bind</code> and prints a warning naming D1 and the identifiable third party in the recording. The publish address is bound on the host side of the port mapping; a container that listens on <code>0.0.0.0</code> internally is not a D1 violation, and a wrapper MUST NOT report it as one.
<code>--bind ADDR</code>	127.0.0.1	
<code>--foreground</code>	off	Streams container logs instead of detaching
<code>--wait-healthy N</code>	<code>\$WORKSHOP_STACK_WAIT (30)</code>	Seconds to wait for <code>/api/health</code> through the published port — never against the container's internal address, which would pass while the mapping is broken

Builds on demand. A missing `$WORKSHOP_BOOT_BIN` or server binary is not a failure; it is a reason to run `build.sh` first, exactly as the reference control plane does. A build that then fails propagates its own exit code unchanged.

Idempotent, and guarded twice on purpose. The wrapper asks `workshop-boot status` before acting and returns `0` unchanged if the stack is already up and healthy; and it passes `compose's --no-recreate`

so that a racing second caller cannot tear down a container that is serving. Neither guard alone survives two concurrent callers, which is why both are contracted rather than one. This composes with, and does not replace, the §2.6 lock.

Stale state must not be trusted. Any prior `$WORKSHOP_STATE_FILE` is removed before the stack starts, so the address the health check probes is the one *this* run actually bound. A health check that passes against a stale endpoint is the exact failure this contract’s three-valued design exists to prevent.

FR-025 is observable at startup: answering-provider construction failure must not abort startup. The stack comes up with an unavailable provider and serves browsing and search; `/api/ask` returns 503. `start.sh` prints which state it came up in. (Gate G-CLI-13.)

Exit	Conditions
0	Serving and healthy (or already was)
1	The published port is bound by a different, identifiable process; the compose file is missing or invalid; the configuration is invalid; a container exited non-zero with a diagnosable cause
2	Health never answered within <code>--wait-healthy</code> ; the container runtime is absent or unreachable; the stack’s state could not be determined at all, so starting would have been a guess; <code>\$WORKSHOP_STATE_FILE</code> could not be written

The 2 for “state could not be determined” is deliberate and is the containerised counterpart of the native design’s unreadable-pidfile case: the wrapper refuses to start a stack whose current state it cannot read, and says why, rather than acting on an assumption.

stop.sh

Brings the stack down through `workshop-boot down`, then removes `$WORKSHOP_STATE_FILE`.

Exit 0 when the stack is stopped — **including when it was already stopped**, because the desired state is reached. 1 when containers were found but did not terminate after the runtime's stop grace period followed by a kill. 2 when liveness could not be determined: the container runtime is unreachable, or its state report could not be parsed.

restart.sh

stop.sh then start.sh, with the same arguments forwarded to start.sh. It is **not** a compose restart: a full down/up is what makes the restart honest about picking up a changed compose file or a rebuilt image. Exit is start.sh's exit, except that a stop.sh exit of 1 or 2 short-circuits and propagates unchanged — restarting on top of a stack that could not be stopped would report success over an unknown.

status.sh

The three-state exemplar. Its contract is stated explicitly because “stopped” could plausibly be argued into any of the three:

Exit	Meaning
0	The stack is RUNNING and /api/health answers through the published port. The condition — <i>the service is up</i> — was checked and holds.
1	Checked, and the service is not up. Determined negative — including the DEGRADED case where containers exist but not all are running, which is a determined negative rather than an unknown.
2	Could not determine : the container runtime is absent or unreachable; a container is reported in a state the wrapper cannot classify; the published port is bound by a process whose identity cannot be established; /

Exit	Meaning
	api/health timed out without connecting or refusing.

stop.sh treats “already stopped” as 0 while status.sh treats it as 1. That is deliberate and not a contradiction: stop reports whether the **desired state** was reached; status reports whether the **asserted condition** holds.

THE IMPLEMENTATION DIVERGES FROM THIS TABLE. The table stands; the code must move.

Decided 2026-09-01. DECIDED BUT NOT APPLIED — the fix is one branch in a file outside the deciding agent’s permitted edit set, so it is written down here in full rather than half-made.

- *Measured 2026-09-01*, first execution of this path on record:

```
WORKSHOP_PROJECT_NAME=zzz-definitely-nonexistent bash
workshop/scripts/status.sh; echo "rc=$?"
# STOPPED
# no containers exist for this project
# rc=0
```

The table above requires 1. The measured value is 0. This is a **real divergence**, not an unknown: the run completed and printed a determinate state.

Note what this measurement retires. The workshop’s own pages recorded the STOPPED exit code as “a code reading, not a measurement” and said the path “was NOT executed”. It has now been executed. The code reading was correct; it is no longer the only evidence.

- *Root cause*: platform/orchestration/cmd/workshop-boot/main.go, cmdStatus, the if len(statuses) == 0 branch — it prints STOPPED and return nil, and nil maps to exit 0.
- *The argument for the code, stated fairly*: under the three-valued discipline used everywhere in this tree — 0 fine, 1 a real problem, 2 could-not-determine — a cleanly stopped stack is a **successful measurement of a benign state**, not a fault. Returning 1 for a stack that was deliberately stopped reads as an accusation. On that reading the contract, not the code, is what got this wrong.

- *Why that argument is refused.* Three reasons, and the third is decisive.

1. **status.sh is a predicate, not a health report.** Its question is “*is the service up?*” 0 means the asserted condition **holds**. That is the convention test, grep, diff and cmp all share, and it is what makes status.sh && deploy correct rather than a trap. The three-valued discipline is not violated by it — it is expressed by it: 2 still means the question could not be answered, which is the distinction that discipline exists to protect. The branch immediately beside the defect proves the code already understands this, returning cannotDetermine (exit 2) when the runtime answers but the project state cannot be read, with the comment “*reporting STOPPED here would be a guess dressed up as a measurement.*”
2. **The “benign state” reading was already anticipated and answered.** The 1 row does not mean *fault*; it means **determined negative**, and it already carries DEGRADED for exactly this reason. Nothing is lost by putting STOPPED beside it, because the *kind* of negative is reported on **stdout** — the script’s first token is RUNNING, STOPPED, DEGRADED or UNDETERMINED. Distinctions belong where they can be read; the exit code answers the yes/no.
3. **The current behaviour has produced a documented, load-bearing defect.** As shipped, status.sh exit 0 **cannot distinguish a running stack from a stopped one**. Five separate workshop pages have had to instruct callers *not to branch on the exit code* and to grep stdout instead. An exit code that all of its own documentation tells you to ignore is not a defensible design choice; it is a defect with a workaround written five times. The contract is not what needs amending here.

- *Exactly what must change* (one branch, plus its proof):

1. platform/orchestration/cmd/workshop-boot/main.go → cmdStatus → the len(statuses) == 0 branch: keep fmt.Println("STOPPED") and the explanatory second line, then return a determined-negative error instead of nil, so the wrapper exits 1. It must **not** route through cannotDetermine — a stack that is known to be stopped was determined, not unknown, and collapsing it into 2 would trade one wrong answer for another.

2. **Paired mutation (§1.1, mandatory):** make that branch return `nil` again and assert the gate goes red. Without it the fix proves nothing.
3. `workshop/scripts/status.sh` must pass the boot binary's exit code through unchanged — verify it does not swallow it before assuming step 1 is sufficient.
4. The five workshop pages that currently say “*do not branch on the exit code*” (`docs/limits.md` §7, `docs/manual.md` §5.1 and its §5 table, `docs/faq.md` §8, `docs/quickstart.md`, `docs/training/areas/05-evidence-gates-and-anti-bluff.md`) must be updated **in the same change**, not after it. Until then they are correct about the shipped binary and must not be edited to describe a fix that has not landed.

- *Until it lands:* the divergence is **live and recorded**. `RUNNING` and `STOPPED` both exit 0 in the shipped binary, the workshop pages’ “read stdout, not the exit code” guidance is the correct instruction **for that binary**, and no gate may report this contract row as satisfied.

`status.sh --json` additionally reports index generation, leg health, the answering provider state, and — new with the containerised seam — the **detected container runtime and its version** and the per-container state, so an operator sees degradation before a user does and can tell a sick container from a sick application.

A note on FR-013 and the reference module. `ai_interviewing` defines zero containers, so the reference cannot be, and is not, imitated at the orchestration layer. FR-013 asks that a person familiar with one module can navigate the other; this section satisfies that through the **script names and their exit semantics**, which match, not through the mechanism underneath them. That distinction is stated rather than left implicit, because “mirrors `ai_interviewing`” would otherwise read as a claim that the reference is containerised. It is not.

4.9 `verify.sh` — the aggregation point

Traces: FR-032, FR-033, FR-040, SC-012, SC-013, SC-018.

Runs every gate in this contract set (G-HTTP-*, G-PID-*, G-CLI-*) and aggregates.

Exit	Conditions
0	Every gate ran and passed

Exit	Conditions
1	At least one gate ran and reported a violation
2	At least one gate could not run — and no gate reported a violation

Precedence is deliberate and is the single most error-prone line in this document: if some gates failed (1) and others could not run (2), the aggregate is **1** — a confirmed violation outranks an unknown — and the summary reports both counts separately. The one thing `verify.sh` must never do is report 0 while any gate was skipped. `PREPUSH_STRICT=1` semantics apply: a SKIP is never a PASS.

Output shape (matching the repository’s existing sweep vocabulary):

```
PASS 21  FAIL 0  COULD-NOT-RUN 3  of 24
COULD-NOT-RUN:
  G-CLI-6  embedding backend unreachable (127.0.0.1:11434
connection refused)
...
```

Every gate registers a paired mutation (FR-032, SC-012). `verify.sh --prove-failure` runs each mutation and asserts its gate turns red; a gate whose mutation does not turn it red is reported as **vacuous** and counted as FAIL. Per the constitution: “*a gate that has never been observed failing is not known to work*”, and “*a mutation proof that exercises only sandboxed copies can go green over an instrument that cannot start at all; proofs MUST include at least one case that runs the real entry point end to end.*”

5. Cross-cutting gates

Gate	Assertion	Paired mutation
G-CLI-1	For every command, <code>result.json.state</code> and the process exit status agree across all three outcomes.	Hardcode state: "ok". Must FAIL.
G-CLI-2	Source (size, mtime, inode) unchanged after a	Rewrite the notes PDF in place. Must FAIL.

Gate	Assertion	Paired mutation
	full pipeline run (§2.4).	
G-CLI-3	ffprobe capability probe rejects Playwright's binary (§3.2).	Use ffprobe --version as the probe. Must FAIL.
G-CLI-4	Deleting one chunk output makes coverage report 1 (§4.1).	Tolerate sub-threshold gaps. Must FAIL.
G-CLI-5	Missing accuracy reference ⇒ exit 2, never 0 (§4.2).	Return 0 with wer: 0.0. Must FAIL.
G-CLI-6	Saturated/unreachable embedding backend ⇒ exit 2, live generation untouched (§4.4).	Map backend failure to 1. Must FAIL.
G-CLI-7	add-chapter.sh run twice satisfies J1-J6 (§4.7).	Mint unconditionally. Must FAIL.
G-CLI-8	Incomplete materials ⇒ exit 1, all findings enumerated, nothing published (§4.7).	Report first finding only and publish. Must FAIL.
G-CLI-9	Evidence exists for every run, including exit 2 (§2.5 E1).	Skip evidence on the failure path. Must FAIL.
G-CLI-10	Every unclassified failure maps to 2, including 127 (§1.4).	Remove the ERR trap. Must FAIL — a missing binary will surface as 127.
G-CLI-11	No .github/workflows/*.yml exists anywhere under workshop/ (§2.7).	Add one. Must FAIL.
G-CLI-12	status.sh returns 2 when the port is	Return 1. Must FAIL.

Gate	Assertion	Paired mutation
	bound by an unidentifiable process (§4.8).	
G-CLI-13	With ollama stopped: <code>start.sh</code> exits 0, <code>/api/search</code> serves, <code>/api/ask</code> returns 503 (FR-025).	Abort startup on provider construction failure. Must FAIL.
G-CLI-14	Privacy negative control (FR-024, D-LLM-4): inside the egress-denied namespace, <code>curl --max-time 5 https://example.com</code> MUST fail, and a packet capture across the full 20-answer and 10-refusal runs shows zero non-loopback packets.	Run the same assertions outside the namespace. Must FAIL — otherwise the test proves nothing about the namespace.

G-CLI-15 | **Containers-actually-booted anti-bluff** (§11.4.76(5)): after `start.sh` exits 0, the container runtime reports the compose project's containers **running**, and `/api/health` was answered by one of them. A green control plane MUST imply the infra was up. | Replace `workshop-boot` with a stub that exits 0 and starts nothing — i.e. reproduce `cmd/boot`'s measured behaviour exactly. Must FAIL. This mutation is not hypothetical: it is what the upstream CLI does today, which is why this gate is the one that would have caught the withdrawn `cmd/boot` specification. |

G-CLI-16 | **Consumption, not reimplementation** (§11.4.76(4)): no lifecycle path under `workshop/` invokes `podman`, `docker`, `podman-compose` or `docker compose` to bring the stack up or down; every such transition goes through `workshop-boot`, which reaches the runtime only through the submodule's `pkg/compose`. Read-only diagnostics on a failure path are exempt and MUST be shown to be exempt by the gate, not by assertion. | Add a `podman-compose up -d` fallback to `start.sh` for when `workshop-boot` is missing. Must FAIL — that fallback is exactly the parallel implementation the clause forbids, and it is the shape drift takes. |

G-CLI-17 | `status.sh` against a project with **no containers** prints

STOPPED **and** exits 1 — a determined negative, distinguishable by the exit code alone from RUNNING's 0 (§4.8; the sibling of G-CLI-12, which pins the same script's 2). Reproduce with
WORKSHOP_PROJECT_NAME=<unused-name> bash workshop/scripts/status.sh. | Restore the return nil in cmdStatus's len(statuses) == 0 branch — i.e. reproduce the behaviour measured on 2026-09-01. Must FAIL. **This gate did not exist while the divergence did, which is how the divergence survived**; it is written down *with* the decision rather than after the fix, so the fix cannot land unproven. |

G-CLI-15 and G-CLI-16 are new with the containerised §4.8 and exist because that rewrite created two properties nothing else in this set covers. **Neither has a task yet** — see §5.1.

G-CLI-14's negative control is what upgrades “*we observed no egress*” into “*egress was impossible*.” A configuration flag is not a guarantee, and hostname string matching is not a security boundary — which is why locality is **declared** and then checked against resolved addresses, never inferred.

G-CLI-16 deserves one line of justification, because a gate that greps for a string is normally the weakest kind. This one is not a grep for style: podman-compose up in a lifecycle path is not a smell, it is the §11.4.76(4) violation itself, so the textual presence *is* the property. Its mutation is a realistic one — a well-meaning fallback for a missing binary — rather than a contrived edit, which is what makes the proof worth having.

5.1 Gate → task coverage — stated, not assumed

Reported rather than silently left for a reader to discover, and **not fixed here**: tasks.md is owned by another agent this session, so this section routes the work instead of doing it.

- **No task in tasks.md cites any G-* identifier**, so no gate in this contract is currently claimed by a task by name.
 - **G-CLI-15 and G-CLI-16 are additionally new**, introduced above by the §4.8 rewrite, and could not have been tasked before now.
 - The **paired-mutation inventory count changes**: this section's table is now **16** G-CLI-* rows, not 14. The PASS 21 FAIL 0 COULD-NOT-RUN 3 of 24 sample in §4.9 is illustrative output, not an inventory, and is not a claim about the count.
-

6. Traceability

6.1 Requirements this contract covers

Requirement	Where
FR-001 complete transcript	§4.1 coverage identity
FR-002 timestamps	§4.1 (absolute time = chunk_start + relative, read from the hashed partition, never accumulated — drift cannot compound)
FR-003 uncertainty marked	§4.1 uncertainty
FR-004 measured accuracy report	§4.2
FR-005 speakers or stated inability	§4.1 speakers
FR-006 sources preserved	§2.4 S1–S4, G-CLI-2
FR-007 reassembly + integrity, fail loudly	§2.1 (invokes <code>extract-videos.sh</code>), §4.1 exit table
FR-012 documented start/stop, no manual setup	§4.8
FR-013 conventions of the reference module	§2.1 script names and the path decision beneath the table; §4.8 closing note. Satisfied by name and exit-semantic parity with <code>ai_interviewing/platform/scripts/</code> , not by directory nesting and not at the orchestration layer, where the reference has nothing to imitate.
FR-016 all content types indexed	§4.3 --kinds, §4.4
FR-018 cross-references	§4.5
FR-020 honest degradation	§4.4 exit 2 on saturation; --lexical-only escape hatch
FR-024 no external transmission	G-CLI-14
FR-025 search survives answering outage	§4.8 <code>start.sh</code> , G-CLI-13
FR-026 single repeatable procedure, no code changes	§4.7

Requirement	Where
FR-027 idempotent	§4.3, §4.7 J1–J6, G-CLI-7
FR-028 precise missing-material report, no partial publish	§4.7 FR-028 block, B1–B6, G-CLI-8
FR-029 resumable, progress, no lost work	§4.1 resume + atomic-rename checkpoints
FR-032 every check has a paired mutation	§5, §4.9 --prove-failure
FR-033 unable-to-verify distinguishable	§1 in full
FR-034 no server-side CI	§2.7 N1–N3, G-CLI-11
FR-037 stable identifiers	§4.3 (delegates to passage-contract.md §8)
FR-038 immutable machine layer	§4.1 machine.jsonl immutable; §4.3
FR-039 redaction required before publication	§4.6, §4.7 B5
FR-040 evidence to a versioned location	§2.5 E1–E4, G-CLI-9
SC-001 coverage 100%, gaps explained	§4.1, G-CLI-4
SC-002 ≥ 30 sampled windows, figure published	§4.2
SC-003 5 s timestamp bound	§4.2 companion metrics
SC-004 fresh clone → running in 15 min	§4.8 (build.sh + start.sh); the tutorial itself is quickstart.md, not this contract
SC-011 add a chapter in 30 min hands-on	§4.7
SC-012 100% of checks have a paired mutation	§5, §4.9
SC-013 three states in 100% of checks	§1
SC-014 zero active CI workflows	§2.7, G-CLI-11
SC-016 references survive correction	§4.3, §4.4 (generation discipline)
SC-018 100% of checks write evidence	§2.5, G-CLI-9

6.2 Requirements no contract in this set covers — stated rather than left as a silent gap

Requirement	Why no interface contract covers it	Where it must be handled
FR-030 — quick-start, user guide, operator manual, FAQ	These are documents, not interfaces. The contracts fix the shapes the documents describe.	workshop/docs/deliverables; quickstart.md (owned by another agent this session)
FR-031 — honest statement of what the system cannot do	Partially covered: the boundaries are stated in-band (http-api.md §3.7 corpus , §3.12 measured:false, passage-contract.md §6.5). The user-facing prose is not.	workshop/docs/
FR-035 — four governance carriers in lockstep	A repository governance property, not an interface.	scripts/verify-governance-cascade.sh; the workshop submodule's own carriers
FR-036 — clean tree across repo and every submodule	An operator/commit-hygiene property.	The project commit wrapper; SC-015
FR-041 / FR-042 — WCAG 2.1 AA, keyboard-operable search	A frontend property. The API affordances that make them attainable are contracted (http-api.md §3.6, §6); conformance itself is not.	Frontend contract + the SC-017 automated audit
SC-017 — automated WCAG audit, zero A/AA violations	Same.	Frontend test suite

Nothing else in the 42 FRs / 18 SCs is uncovered.

7. Unverified register

#	Item	Status	Settled by
U1	cp314 wheels for CTranslate2 / faster-whisper install and run on this host. Every command in §4.1 presumes them.	UNVERIFIED	Attempt the project-local venv install. Until it succeeds, <code>transcribe.sh</code> exits 2 at preflight — which is the correct report, not a blocker on this contract. <code>transcribe.sh</code> <code>--sample-seconds 300</code> on the already-extracted 300 s sample.
U2	Real transcription throughput on this CPU. The 40–80 minute figure is an estimate , roughly doubled under current load.	UNVERIFIED	Progress ETA is derived from the measured rate, never from this estimate (§4.1). The same calibration run. Until it closes, <code>add-chapter.sh</code> must not publish without <code>verify-accuracy.sh</code> having run (B2) — the accuracy gate is what would catch a transcript of nothing.
U3	That the recording contains intelligible speech in a known language. Nobody has listened and no ASR has run.	UNVERIFIED	

#	Item	Status	Settled by
U4	Idle embedding latency; the -- timeout-ms 5000 default and the SC-006 budget assume it.	UNVERIFIED	Time ten consecutive searches in one warm session against an idle backend, after the running lumen index -- force rebuild completes.
U5	SC-007/SC-008 retrieval quality and the relevance floor. No corpus exists yet.	UNVERIFIED	Build the ≥20-query benchmark set after Chapter 1 is indexed, then calibrate.
C-U6	Container orchestration. plan.md treats containerisation as new work (the reference module defines zero containers), and constitution §11.4.76 requires va sic-digital/containers as the sole orchestration layer — which is not a submodule of this tree. §11.4.161 mandates rootless podman; the host has podman 5.7.1 rootless and no docker — that half stands, measured.	UNRESOLVED, not merely unverified → RESOLVED 2026-09-01. Original text kept struck through, not deleted (see the withdrawal note below the table).	An operator decision on whether vasic-digital/containers is added as a submodule. This contract therefore specifies no container commands. sta rt.sh/stop.sh are the documented FR-012 interface and run the built binary natively, as <code>ai_interviewing</code> does. The submodule is declared and populated; §4.8 now specifies the

#	Item	Status	Settled by
			containerised control plane. Adding a bespoke container stack is still forbidden by §11.4.76(4) — that clause of the original entry was never wrong.
C-U7	<code>ai_interviewing's search.component.ts</code> defect (http://api.md§2.2) exists independently of this feature and is out of scope here. Whether to fix it is an operator decision.	Noted	An operator decision; it is not a dependency of this feature.

7.1 Withdrawal — C-U6's “not a submodule of this tree”

Recorded explicitly because this contract set treats a stale claim as something to **withdraw**, never to quietly overwrite; a reader must be able to tell a corrected fact from one that was never wrong.

The withdrawn claim	“constitution §11.4.76 requires vasic-digital/containers as the sole orchestration layer — which is not a submodule of this tree”, and its consequence, “this contract therefore specifies no container commands ... start.sh/stop.sh ... run the built binary natively, as ai_interviewing does”.
What was believed, and why it was reasonable	Written 2026-08-31. At that moment .gitmodules declared no vasic-digital/containers gitlink.

§11.4.76(2) makes the submodule a hard prerequisite for any containerised workload, so with the prerequisite unmet the only §11.4.76-legal position was to specify nothing containerised and to escalate the gap as an operator decision. The reasoning was sound on the tree as it then stood.

```
2026-09-01. git config
-f .gitmodules --get-regexp
containers →
submodule.submodules/
containers.path submodules/
containers and ...url
git@github.com:vasic-digital/
containers.git.submodules/
containers is populated (pkg/
compose, pkg/health, pkg/runtime
present). git ls-tree HEAD
submodules/containers → gitlink
pinned at 4dab992. scripts/
verify-governance-cascade.sh
classifies 9 declared submodules
from evidence — 7 owned,
including containers.
```

What is measured now

During this feature's own Phase 1 work — after the C-U6 row was written, before this withdrawal. The claim was true when made and is false now; both facts are on the record.

When it changed

Two things the original row got right and that remain binding: (a) ai_interviewing defines **zero** containers, so “run it through the containers the same way” cannot mean copying the reference module — the reference is a naming and navigability precedent (FR-013), not a container precedent; (b) a

What the withdrawal does NOT overturn

bespoke Containerfile or hand-rolled compose stack is a §11.4.76(4) violation, and remains forbidden.

Consequence for this contract

§4.8 is rewritten. The control plane is **containerised**, launching through the workshop-boot adapter over submodules/containers' pkg/compose/pkg/health/pkg/runtime Go API — **not** through its cmd/boot CLI, which §4.8.0 measures as unusable. Native launch is no longer specified anywhere in this document.

The architecture is decided and is not reopened by this document. The operator requires container operation and §11.4.76 mandates the submodule; those point the same way, so there is no trade-off left to weigh. The residual §11.4.161 constraint (rootless podman, no docker) was never withdrawn and is honoured by the submodule's podman-first backend selection.

Measured, not assumed, and therefore not in this register: the ffp/Playwright symlink trap; the absence of any ASR engine and of any generative model; the recording's duration (6928.75 s), dual-mono audio and eight long silences; all embedding servers exhausted from a backend whose health_check said OK 2 ms earlier; the half-written readable index during rebuild; FTS5 prefix latency at p95 9.58 ms; and the 20.16 / 11.05 / 0.10 s spread across three identical embed calls.